Engineering Applications of Artificial Intelligence 160 (2025) 111713

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Research paper

A general graph learning based framework for job offer recommender system

Ghislain Wabo Tatchum [a,*], Armel Jacques Nzekon Nzeko'o [a,b,1], Thomas Messi Nguele [a,1], Xaveria Youh Djam [a,1]

[a] Departmentof computer sciences, University of Yaounde I, BP 812, Cameroon
[b] Sorbonne University, IRD, UMI 209 UMMISCO, Bondy, F-93143, France

ARTICLE INFO

Keywords:
Recommender systems
Graph learning
Job offers
Machine learning
Graph embedding

ABSTRACT

In the digital age, job markets have undergone a transformation, becoming supply and demand for jobs through e-recruitment platforms. These e-recruitment platforms generally contain recommender systems to ensure easy filtering of large quantities of job offers and/or jobseeker profiles. Recommender systems play a major role in the fight against unemployment due to their strong presence in job recommendation platforms, by reducing the time taken to search for jobs and profiles. The use of graphs in recommender systems offers a more natural and intuitive technique for representing the interactions between jobseekers and job offers and thus providing good recommendations. In order to do this, these recommender systems rely on different types of information, such as the characteristics of the description of the offers/applicants, the history of interactions between profiles and offers, location, time, profile level and the number of experiences of the profiles. Existing studies that have tackled this subject generally use only one or two of these pieces of information, which limits their performance. They therefore have a common procedure which consists of constructing the graph from the job offers or jobseekers and computing recommendations on the graph, but with the limitation that the graph construction techniques are very specific to the dataset and therefore difficult to reproduce on other data. In additional, these graphs are mainly exploited using random walks algorithms and very few have explored graph learning technologies. In this paper, we propose a general framework based on graph learning for job recommender systems. This framework can be used to construct graphs (job offer-oriented graph, job seeker-oriented graph, bipartite graph of job offers and job seekers) from characteristics based on the description of the offer or profile, on interactions between job offers and job seekers, on temporal dynamics, on location, on the level of job seekers and on the number of experiences of job seekers; with the possibility of varying the weight of each characteristic to adapt to different data sets. It can then be used to learn graphs for embedding the various graphs constructed and finally to compute the recommendations based on graph learning or graph embeddings. We experimented our framework on Nigham and LinkedIn datasets, using Node2Vec, Graph Sample and Aggregation (GraphSage) and Graph Convolutional Networks (GCN) as graph embedding techniques, Decision Trees, eXtreme Gradient Boosting (XGBoost), Support Vector Machine (SVM), Bidirectional Encoder Representations from Transformers for Recommendation (BERT4Rec) and GCN as top-N recommendation algorithm and Accuracy, Precision, F1-score and Area Under the Curve (AUC) as evaluation metrics. The results obtained show that compared with standard graph-based recommendation techniques (PageRank and K-Nearest Neighbors) we have improvements of 7%, 5%, 11% and 11% respectively for accuracy, precision, F1-score and AUC, and compared with previous work an increase of up to 54% for F1-score.

1. Introduction

The advent of information and communication technologies (ICTs) has transformed the traditional job supply and demand market with online job publication and recruitment platforms: this is e-recruitment. E-recruitment platforms have had a positive impact on the global job market in general and the African job market in particular (Hjort and Poulsen, 2019). In addition, users of e-recruitment platforms see their probability of being re-employed increased and recruiters save

* Corresponding author.
E-mail addresses: ghislain.wabo@facsciences-uy1.cm (G.W. Tatchum), armel.nzekon@facsciences-uy1.cm (A.J.N. Nzeko'o), thomas.messi@facsciences-uy1.cm (T.M. Nguele), xaviera.kimbi@facsciences-uy1.cm (X.Y. Djam).
[1] These authors contributed equally to this work.

https://doi.org/10.1016/j.engappai.2025.111713
Received 25 January 2025; Received in revised form 3 June 2025; Accepted 5 July 2025
Available online 24 July 2025
0952-1976/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

a huge amount of time in the recruitment process (Suvankulov et al., 2012). But with the huge amount of data on these platforms, searching for information has become tedious, hence the need to integrate job offer recommendation systems into these platforms. Job recommendation systems are a specific form of information filtering, consisting of proposing job offers that best correspond to the profiles of job seekers.

Job recommendation systems were born with the emergence of job search engines (Joy et al., 2021; Akter et al., 2023) and over the last decade, the penetration rate and use of the Internet for job searches, particularly job search engines, has risen sharply worldwide. The most emblematic case is the Indeed[2] platform, with more than 250 million visitors per month (Jiang et al., 2019a). We also have others platforms such as LinkedIn[3] and CareerBuilder.[4]

Furthermore, research has shown that jobseekers who have used the Internet see their probability of being re-employed in the next 12 months rise from 7.1% to 12.7%. Still with the aim of showing the benefits of job recommendation platforms, Freire and de Castro (2021) claim that job search engines save between 6 and 9 h per recruitment and recruit 4 times more than usual; this is a significant financial and time saving for companies, and a positive social impact in the fight against unemployment. What is more, the turnover of job search engines has risen sharply in recent years, thanks on the one hand to partnerships with companies and on the other hand to growth in advertising revenue due to the increase in the number of visitors.

Platforms integrating recommendation systems for finding transitive associations are generally implemented using a graph-based model. These are recommendation graphs, which are not only easy to interpret but also offer the possibility of directly implementing numerous algorithms in different domains. The use of graphs in job recommendation systems therefore offers a much more instinctive way of representing the interactions between jobseekers and job offers and thus providing jobseekers with the most appropriate offers for their profiles.

In order to make good recommendations, these recommendation systems rely on different types of information in the dataset, such as the characteristics of the description of job offers and job seekers, the history of interactions between profiles and offers, location, time, level of profiles and number of experiences of profiles. Job offer recommendation systems have been addressed by many works in the literature (Chou and Yu, 2020; Giabelli et al., 2021; Luo et al., 2019; Dave et al., 2018; Wang et al., 2020...), among them are others interested in the use of graphs (Luo et al., 2019; Dave et al., 2018; Wang et al., 2023; Zheng et al., 2023...). These existing works, which deal with the use of graphs to recommend job offers, do not cover all this informations, focusing only on those that primarily describe them, which limits their performance. Therefore, they have a common procedure which consists of constructing the graph from job offers or job seekers and calculating the recommendations on the graph. In addition, each propose various techniques for constructing graphs, but these techniques are specific to the data they have used because they are based on characteristics specific to these datasets, and therefore cannot be applied to other data. Finally, these graphs are mainly exploited using random walk algorithms and very few have explored graph learning technologies.

With the aim of providing more systemic techniques for constructing graphs and exploring graph-based job recommendation in a more complete way, we propose in this paper a general framework based on graph learning for recommender systems (GLFJRec). It is a framework that firstly allows graphs to be constructed from the main characteristics of job offers and job seekers, where the weights of the characteristics can vary so that they can be adapted according to the dataset. It can then be used for graph learning to obtain a graph embedding representation, and finally it can also be used for graph learning to make recommendations directly on the graph or from graph embeddings.

To do this, we construct three graphs (job offer-oriented graphs, job seeker-oriented graphs and bipartite graphs of job seekers and job offers) from characteristics based on the description of the job offer or job seeker profile, the interactions between job offers and job seeker profiles, the temporal dynamics, the location, the level of education of the job seeker profiles, and the number of experiences of the job seeker profiles. We assign weights to these characteristics $\alpha, \beta, \gamma, \delta, \omega$, which take their values in the interval $[0, 1]$ with for condition the sum must be equal to 1. This assignment can be made by manual parameterization, or by letting the framework test parameter combinations to determine the optimum setting. For the embedding we used GraphSage, Node2Vec, GCN and for the learning models Decision Tree, XGboost, SVM, BERT4Rec and GCN for the computation of the recommendations. The experiments were carried out on two datasets, Nigham and LinkedIn, and the metrics used to evaluate the results were accuracy, precision, F1-score and AUC. The results obtained were compared with those of recommendation methods based on graph walks (KNN and PageRank), with previous work (Zhang and Xu, 2024; Yang et al., 2022) and we also ran the GLRec framework (Wu et al., 2024) on our data and compared it with our framework.

The rest of this paper will be structured as follows: in Section 2 we will present previous work on graph-based job offer recommendation, in Section 3 we will present the approach used to build the framework for constructing the job offer graph, in Section 4 we will present the experiment and the results, and in Section 5 a conclusion and outlook.

2. Related work

In this section, we will first present the work already carried out in the field of job offer recommendation, then that carried out in the framework of graph-based recommendation and finally that which uses graph learning techniques for job offer recommendation.

2.1. Job recommendations

The birth of e-recruitment platforms with the advent of ICTs has turned the interest of many researchers to job recommendation systems.

Thus, work has been proposed such as that of Jiang et al. (2019b) which is based on observing the clicking behavior of a large number of users in a major job search engine, and so they develop a set of features that reflect the clicking behavior on this search engine, of individual job seekers. And they use the EM algorithm for joint estimation of the parameters of the classification and prediction process. We also have those from Nigam et al. (2019), who are also looking at recommender systems, introducing a new machine learning model that uses job candidate preference over time to incorporate the dynamics associated with a highly volatile labor market. In addition to this, this new approach takes into account several other aspects such as: generating incidental referrals, solving the cold start problem for new jobs and new candidates. They use competencies as embedded features, allowing candidates' skills to be extended. The model they propose has been deployed on a real-world recommendation system, and achieved the best performance according to the click-through rate metric. Performances are obtained when the model is based on machine learning notably long and short term memory neural networks.

Chou and Yu (2020) are developing a system which, using machine learning and artificial intelligence technologies, analyzes trends in online discussions and electronic CVs submitted by candidates, and obtains candidate characteristics such as personality traits, skills and competitiveness; these characteristics are then used to generate a list of talents for companies. Mhamdi et al. (2020) propose a system that recommends to a user interested in an offer, the n offers closest to it. They start by collecting offers from websites, then prepare the offers by extracting relevant features such as the offer title and technical tasks.

[2] https://www.indeed.com/.
[3] https://www.linkedin.com/.
[4] https://www.careerbuilder.com/.

Based on these characteristics, they measure the proximity between offers and form clusters using the K-means algorithm. In this way, they recommend the n closest offers in the same cluster to a user who has viewed that offer.

2.2. Graph based job recommendation

Some authors (Shalaby et al., 2017; Giabelli et al., 2021), with the aim of providing better recommendation performance and making the results more interpretable, have carried out work approaching graph-based job offer recommendation from several aspects.

We have the work of Shalaby et al. (2017) which tackles the problems of scalability and cold start in job recommendations by proposing a graph-based recommendation system. They start by building a graph of job offers, the nodes of which are the job offers. The stops are created based on co-actions (clicks and joint applications) between the job offers and also on the similarity of the content of the offers. Once the graph has been constructed, they use the PageRank algorithm and a customized version of PageRank to calculate the top-N recommendation.

In the work of Giabelli et al. (2021), they are interested in a system that represents job offers in a graph database. They start by mining online job market data using distributional semantics, a knowledge-based representation and a measure of skill relevance. They then organize the aforementioned resources in the form of a graph database to perform queries across graphs and finally present skills2job, a recommendation system that exploits the resources developed above to suggest the most appropriate occupations based on the user's skills.

Previous studies have approached the recommendation of job offers using recommendation graphs, but they propose graph construction techniques specific to their data, and calculate the recommendation directly on graphs using graph walking algorithms.

2.3. Graph learning based job recommendation

With the aim of extracting as much information as possible from job recommendation graphs, a number of studies (Luo et al., 2019; Dave et al., 2018; Wang et al., 2020, 2023; Zheng et al., 2023; Zhang et al., 2023; Han et al., 2024) have proposed learning from graphs, and thus increasing job recommendations.

Thus, the work of Dave et al. (2018) focuses on the recommendation of job offers and skills, by constructing three graphs (job transition graph, job skills graph, job skills co-occurrence graph). Then, by learning the graphs, they provide a vector representation (embedding) of the graphs (job offers and skills) and from the cosine similarities between the representations of job offers and skills, they make a top-N recommendation. In the same vein, the work of Luo et al. (2019) attempts to provide a latent representation of users and job offers that can take into account the similarity between offers, location and the career development of users. They construct a graph of job offers and, using graph embedding techniques, provide a representation of offers and profiles. From these representations, they will use a similarity measure to make top-n recommendations.

In the same way, Han et al. (2024) addresses the issue of user preference drift in job recommendation systems with their BISTRO framework. It combines behavioral and semantic information through fusion learning to model user preferences more effectively. The process involves three steps: interaction segmentation, fine-grained preference extraction, and personalized job recommendations, using hypergraph wavelet learning and a recurrent neural network. A key limitation is that the approach only relies on semantic and behavioral data from job offers and users, without utilizing additional data like social interactions or professional network-based recommendations, which could enhance the system's ability to capture evolving user preferences.

In these two works, graph learning is only used for graph embedding, and not for recommendation, authors have also thought of applying graph learning to the calculation of top-N recommendations. This is the case of Wang et al. (2020) who provide a session-based job offer recommendation system combining domain knowledge and graph neural networks. They propose a system which, based on a set of previous clicks in a user's session, provides the probability that they will click on other job offers not yet clicked on, and from these probabilities, they make a top-n recommendation. On the other hand, the work of Wang et al. (2023) proposes an approach based on graph learning with two ways of recommending. The first is to perform graph embedding using the Node2Vec, GCN, GraphSage algorithms, etc. to provide a numerical representation of the nodes. The idea being to capture the general structure of the graph in this representation. Then, using these representations, assimilate the recommendation problem to the classification of node embeddings or link prediction using algorithms such as GCN, SVM, BERT4Rec, LR. And the other way is to learn the graph directly for the calculation of recommendation with the GCN. Still in the context of learning graphs for job offer recommendations, a framework based on Large Languages Models (LLM) has been proposed (Wu et al., 2024) GLRec for a better understanding of graphs. It proposes a heterogeneous graph of both job offers and job seekers; the experiments are carried out on a dataset that they have constructed and collected from a Chinese job offer publication platform that is unfortunately not available.

The use of graphs in job recommendation systems has greatly improved the performance of these systems. However, existing work only takes certain criteria into account when constructing graphs and neglects others that may prove decisive for the recommendation process in certain cases and, above all, this makes the construction of graphs specific to their dataset. Also, in each work it is difficult to compare the different approaches to calculating recommendations on their graphs because they only propose one or two approaches for calculating recommendations. This is why in this work we propose a general framework based on graph learning for job offer recommendation, which will make it possible to choose the criteria to be taken into account in graph construction, to choose which type of graph to construct (job offer-oriented graph, user-oriented graph and bipartite graph) and also which approaches to use for recommendation.

3. General job recommendation framework

In this section, we will start with an overview of the framework, followed by a presentation of the job offer-oriented, user-oriented and bipartite graph construction approaches.

3.1. General architecture of the framework: GLFJRec

To make the most of the information available in the dataset and propose a more generic model for recommending job offers, we built the GLFJRec framework. How it works is shown in Fig. 1.

It begins with a pre-processing step on the data, in which the dataset is cleaned so as not to extract unnecessary elements (e.g. the tags that are in the job descriptions). In data science, the quality of the data has a direct impact on the performance of the algorithms based on this data, so it is essential to have good quality data (Uysal and Gunal, 2014; Wisdom and Gupta, 2016). However, in general, the raw data collected often contains numerous faults, hence the pre-processing stage. For textual data such as job descriptions and job seeker profiles inspired by previous work (Perkins, 2010; Afandi and Isnaini, 2024) we removed stopwords, tokenised and stemmed (Patel et al., 2024) the text. And for data such as dates and numbers of experiments, we removed missing lines and inconsistent values.

This is followed by the extraction of input functions and function parameters (Patel et al., 2024). The input functions here are characteristics or criteria extracted from the dataset that will enable us to construct the graphs. These are, among others, for the job offer oriented graph, the characteristic based on the content of the offer (description

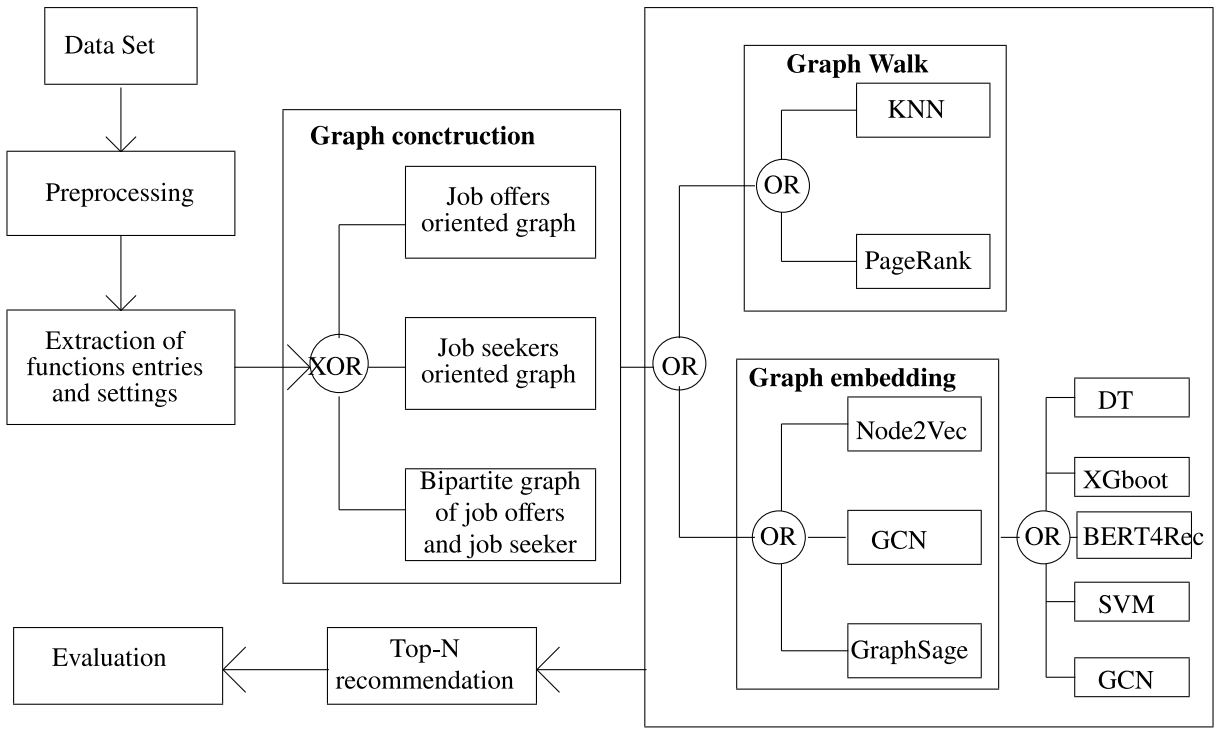

Fig. 1. Diagram of the GLFJRec framework workflow from dataset to recommendation and evaluation of recommendations.

of the offer), the characteristic based on the common interactions carried out on the offers (here we are talking about the clicks and applications carried out by the same profile on two offers), the characteristic based on the temporal dynamics (here we are talking about the time taken since the publication of the offer) and the characteristic based on the location (the place where the offer profile will be exercised). For the job seeker oriented graph, in addition to the characteristics based on profile content, common interactions between profiles and profile location, we have the characteristics based on the level of the job seeker and finally the characteristic based on the number of work experiences already carried out. These features are extracted from pre-processed datasets and supplied to the graph construction functions.

We then proceed to construct the graphs. This step is presented in more detail in the next sub-section. Once the graph has been constructed, we have the option of either learning the graph directly for recommendation using the GCN model, or first learning the graph for graph embedding using the Node2Vec, GrapheSage and GCN techniques, and then using the representation of the graphs to learn for recommendation using the DT, SVM, XGboost, BERT4Rec and GCN models. In addition, we also carry out a classic recommendation based on graph traversal with the KNN and PageRank algorithms, to which we will then compare the performance of the results based on graph learning.

Once the recommendation has been made using one or more of the framework's processes, it also offers the possibility of evaluating the recommendation, on the one hand, by comparing the various recommendation processes, but also, further down the line, by comparing different graph embedding methods.

3.2. Graph construction

In this sub-section, we present the algorithms for constructing the various graphs, in particular the job offer-oriented graph, the user-oriented graph and the bipartite graph.

3.2.1. Job oriented graph

A job offer oriented graph G is the 3-uplet (J, E, W), where J is a set of job offers $\{ J_1, J_2, J_3, \dots, J_n\}$, E is the set of undirected edges $e_{i,k}$ linking a job offer i to another k, and W is the set of weights $w_{i,k}$ of the edges $e_{i,k}$. We define a function Ψ which will determine the weight of the edges defined as follows:

$$\Psi : \mathbf{F}(J, J) \longrightarrow [0; 1]$$
$$\psi \longmapsto \alpha\psi_1(J_i, J_k) + \beta\psi_2(J_i, J_k) + \gamma\psi_3(J_i, J_k) + \delta\psi_4(J_i, J_k)$$

where:

- $\psi_1(J_i, J_k)$ is a content based criteria compute as cosine similarity between J_i and J_k description. This cosine similarity is calculated on the vector representation of job offers obtained through class-oriented text vectorization.
- $\psi_2(J_i, J_k)$ is the common interaction based criteria compute as the ratio of simultaneous number of co actions between J_i and J_k. These actions are applications, exercises, clicks and additions.
- $\psi_3(J_i, J_k)$ is the temporal dynamic based criteria compute as the date comparison function between J_i and J_k. We assume that two job offers are closer together if they are published at the same time. This allows us to take into account factors such as technological evolution and the highly volatile nature of job offers. We have defined it as follows:

  - $\psi_3(J_i, J_k) = 0$ if Δ date $(J_i, J_k) \geq 360$ days;
  - $\psi_3(J_i, J_k) = 0.25$ if Δ date $(J_i, J_k) \geq 180$ days;
  - $\psi_3(J_i, J_k) = 0.5$ if Δ date $(J_i, J_k) \geq 90$ days;
  - $\psi_3(J_i, J_k) = 0.75$ if Δ date $(J_i, J_k) \geq 30$ days;
  - $\psi_3(J_i, J_k) = 0.75$ if Δ date $(J_i, J_k) \leq 29$ days;

  The interval divisions above were obtained by clustering with the k-means algorithm on job offers in the same fields with $K = 5$ (the K value with the highest silhouette score), and by analyzing the different clusters it emerged that the closer the job offers were in time, the closer the terms to describe them were too.

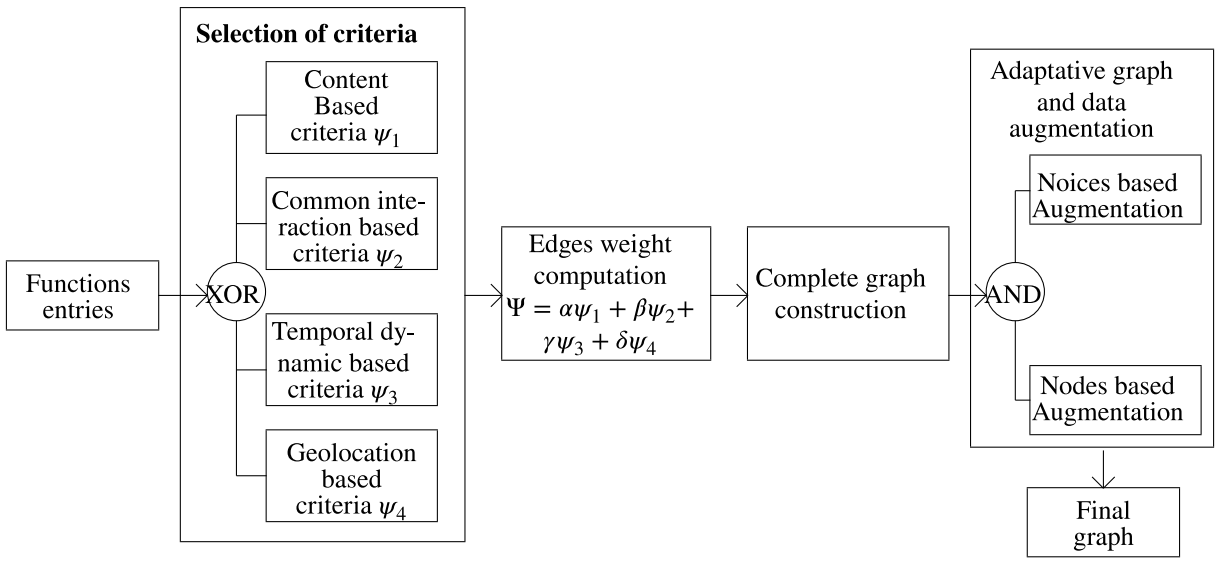

Fig. 2. Diagram of the construction of the job offer-oriented graph based on the characteristic functions of job offers.

- $\psi_4(J_i, J_k)$ is the geolocation based criteria. We have assumed that a job offer in the same region as the one the user is interested in will be preferred to one far away. As a result the location comparison function between J_i and J_k define by :
  - $\psi_4(J_i, J_k) = 1$ if J_i and J_k are in the same location;
  - $\psi_4(J_i, J_k) = 0$ if J_i and J_k are not in the same location;

for all $1 \leq i, k \leq n$. α, β, γ and δ are parameters that define the weight each criterion will have in relation to the equation. If all criteria have the same weight, we will have the same value $\alpha = \beta = \gamma = \delta = 1$ for all parameters. By normalizing the weights of each criterion, we get $\sum(\alpha, \beta, \gamma, \delta)$ =1 and therefore $\alpha = \beta = \gamma = \delta$ =0.25. However, for the framework to be generalizable to all datasets in which certain criteria may not be present, the value of one or more of the parameters can be zero, thus increasing the weight of the others. We then have α, β, γ and $\delta \in \{ 0, 0.25, 0.5, 0.75, 1 \}$. These parameters are used to decide which criteria will be taken into account when constructing the graph, and how much weight they will have.

Fig. 2 shows the process of constructing the job offer-oriented graph. At the end of the graph construction process, inspired by previous work (Jin et al., 2023) we proceed to an adaptive graph augmentation based on noise (here is considered as noise the arrests having a weight lower than the third quartile of the weights, so the adaptive graph augmentation based on noise consists in deleting these edges considered as noise) and based on nodes (the adaptive augmentation based on nodes consisted in deleting nodes that were outside the lower part of the node centrality whisker box and therefore characterized as low centrality). We note that in the GLFJRec framework, it is possible to set threshold values for adaptive graph augmentation, or even vary them to obtain optimal values.

If we take n to be the number of job offers, then we can write Algorithm 1, which constructs the job offer graph. From this algorithm we see that for each job offer, a set of operations is performed on the pairs of job offers, giving us a time complexity of the order of $\Theta(n^2+n)$. An increase in the number of job offers will lead to a quadratic increase in execution time.

Algorithm 1 GraphJobOfferFactory

Require: df: xlsx;
Ensure: G: graph;
$List_{desc} \leftarrow$ Vectorize (df[description]);
$List_{user} \leftarrow df[users]$
$List_{date} \leftarrow df[date]$;
$List_{loca} \leftarrow df[location]$;
while $J_i \in \{J_1, J_2, J_3, ...J_n\}$ **do**
  while $J_k \in \{J_1, J_2, J_3, ...J_n\}$ **do**
    $\psi_1(J_i, J_k) \leftarrow Cosine_Similarity(List_{desc}[J_i], List_{desc}[J_k])$;
    $\psi_2(J_i, J_k) \leftarrow Intersection(List_{user}[J_i], List_{user}[J_k])/n$;
    $\psi_3(J_i, J_k) \leftarrow Date_Compare(List_{date}[J_i], List_{date}[J_k])$;
    $\psi_4(J_i, J_k) \leftarrow Location_Compare(List_{loca}[J_i], List_{loca}[J_k])$;
    $W_{J_i,J_k} \leftarrow \alpha\psi_1(J_i, J_k) + \beta\psi_2(J_i, J_k) + \gamma\psi_3(J_i, J_k) + \delta\psi_4(J_i, J_k)$;
    if $W_{J_i,J_k} \geq 0.60$ **then**
      G.add (J_i, J_k, W_{J_i,J_k});
    end if
  end while
end while
while $J_k \in \{j_1, j_2, j_3, ...j_n\}$ **do**
  if $Centrality(J_k) \leq \mu$ **then**
    G.remove (J_k);
  end if
end while
return G;

3.2.2. Job seeker oriented graph construction

A job seeker oriented graph G is the 3-uplet (U, E, W), where U is a set of users (profiles) $\{ u_1, u_2, u_3, \ldots, u_n \}$, E is the set of undirected, weighted $e_{i,k}$ edges linking one user i to another k, and W is the set of $w_{i,k}$ weights of the $e_{i,k}$ edges. We define a function Ψ which will determine the weight of the edges defined as follows:

$$\Psi : \mathbf{F}(U, U) \longrightarrow [0; 1]$$
$$\psi \longmapsto \alpha\psi_1(U_i, U_k) + \beta\psi_2(U_i, U_k) + \gamma\psi_3(U_i, U_k) + \delta\psi_4(U_i, U_k) + \omega\psi_5(U_i, U_k)$$

where:

- $\psi_1(U_i, U_k)$ is the profile content based criteria compute as cosine similarity between U_i and U_k user profile description.
- $\psi_2(U_i, U_k)$ is the common interaction based criteria compute as the ratio of simultaneous number of co actions between U_i and U_k. These actions are applications, exercises, clicks and additions.
- $\psi_3(U_i, U_k)$ is the criterion based on the education level of users U_i and U_k. We assume that two user profiles with the same level of

education will be interested in similar job offers. For this reason, we defined a dictionary of the different educational levels we had, and inspired by the work of Pelánek (2020), we proposed a similarity measure between the different levels present in the vocabulary, taking into account the hierarchy that exists between them.

- $\psi_4(U_i, U_k)$ is the criterion based on the professional history of user profiles. This assumes that two users with similar work histories will have similar interests. The professional history similarity here is based on the number of professional experiences each user has. And for each experience, the sum of the values is 1 if it is the same field of activity and 0 otherwise.
- $\psi_5(i,k)$ is the geolocation based criteria. We have assumed that users living in the same region are likely to have the same professional interests. This criterion was therefore calculated as follows:

  - $\psi_5(i,k) = 1$ if U_i and U_k are in the same location;
  - $\psi_5(i,k) = 0$ if U_i and U_k are not in the same location;

for all $1 \leq i, k \leq n$.

$\alpha, \beta, \gamma, \delta$ *and* ω are parameters that define the weight each criterion will have in relation to the equation. If all criteria have the same weight, we will have the same value $\alpha = \beta = \gamma = \delta = \omega = 1$ for all parameters. By normalizing the weights of each criterion, we get $\sum(\alpha, \beta, \gamma, \delta, \omega)$ =1 and therefore $\alpha = \beta = \gamma = \delta = \omega$ =0.2. However, for the framework to be generalizable to all datasets in which certain criteria may not be present, the value of one or more of the parameters can be zero, thus increasing the weight of the others. We then have α, β, γ and $\delta \in \{\ 0, 0.2, 0.4, 0.6, 0.8, 1\ \}$.

Fig. 3 shows the process of constructing the job offer-oriented graph. In the same way as before, at the end of the graph construction process inspired by previous work (Jin et al., 2023) we proceed to an adaptive graph augmentation based on noise (here is considered as noise the edges having a weight lower than the third quartile of the weights, so the adaptive graph augmentation based on noise consists in deleting these edges considered as noise) and based on nodes (the adaptive augmentation based on nodes consisted in deleting nodes that were outside the lower part of the node centrality whisker box and therefore characterized as low centrality).

Note that in the GLFJRec framework, it is possible to set threshold values for adaptive graph augmentation, or even vary them to obtain optimal values.

We present below the algorithm for constructing a job offer-oriented graph. Given n the number of job offers, this algorithm has a time complexity of $\Theta(n^2 + n)$. This implies that the execution time of this algorithm is a function of the square of the number of users. Hence, as the number of users increases, the execution time will quadratically increase.

3.2.3. Bipartite graph of job offer and job seeker construction

We call a bipartite graph the graph that combines both job offers and users. This graph shows not only the links between job offers and profiles, but also the links between users and job offers. Fig. 4 illustrates the construction of this graph.

Algorithm 3 constructs a bipartite graph of job offers and user profiles as follows. It assumes that the individual job offer and user graphs have already been constructed. So, if we consider n as the number of job offers and m as the number of users, the time complexity of this algorithm is of the order of $Theta(nm)$. This means that the execution time increases as a function of the maximum between n and m.

Algorithm 2 GraphJobSeekerFactory

Require: Udf: xlsx;
Ensure: G: graph;
$List_{profil} \leftarrow$ Vectorize (Udf[profile]);
$List_{job} \leftarrow Udf[users[job]]$
$List_{stl} \leftarrow Udf[studylevel]$;
$List_{hist} \leftarrow Udf[history]$;
$List_{loca} \leftarrow Udf[location]$;
while $U_i \in \{U_1, U_2, U_3, ...U_n\}$ **do**
  while $U_k \in \{U_1, U_2, U_3, ...U_n\}$ **do**
    $\psi_1(i,k) \leftarrow Cosine_Similarity(List_{profil}[U_i], List_{profil}[U_k])$;
    $\psi_2(i,k) \leftarrow Intersection(List_{job}[U_i], List_{job}[U_k])/n$;
    $\psi_3(i,k) \leftarrow StudyLevel_Compare(List_{stl}[U_i], List_{stl}[U_k])$;
    $\psi_4(i,k) \leftarrow History_Compare(List_{hist}[U_i], List_{hist}[U_k])$;
    $\psi_5(i,k) \leftarrow Location_Compare(List_{loca}[U_i], List_{loca}[U_k])$;
    $W_{ik} \leftarrow \alpha\psi_1(i,k) + \beta\psi_2(i,k) + \gamma\psi_3(i,k) + \delta\psi_4(i,k) + \omega\psi_5(i,k)$;
    if $W_{ik} \geq 0.60$ **then**
      G.add (i, k, W_{ik});
    end if
  end while
end while
while $U_k \in \{U_1, U_2, U_3, ...U_n\}$ **do**
  if $Centrality(U_k) \leq \nu$ **then**
    G.remove (U_k);
  end if
end while
return G;

Algorithm 3 BipartiteGraphFactory

Require: G_1: Graph of Jobs;
G_2 : Graph of Users;
Ensure: G: graph;
G.add(G_1, G_2);
$List_{job} \leftarrow Udf[users[job]]$
while nodes $J_k \in G_1$ **do**
  while nodes $U_k \in G_2$ **do**
    if $J_k \in list_{jobs}[U_k]$ **then**
      G.addEdges (U_k, J_k);
    end if
  end while
end while
return G;

3.3. Illustration of construction

Let the Table 1 contain the input functions for constructing the job offer graph, extracted from the dataset.

The descriptions column contains the textual description of the profile being sought, the users column contains all the users who have interacted with the job in question, and the date and location columns are respectively the date and location where the job is being sought. The job descriptions are vectorized using the OWDC-Doc2Vec (Tatchum et al., 2024) approach and the ϕ_1 function is calculated using the cosine similarity between the two vector representations of the descriptions. For the example of the table $\phi_1(J_1, J_5) = 0.537$.

As for the function ϕ_2, it will be the fraction of the number of users that two offers have in common over the total number of users, in our example if the number of users is 4 we will have $\phi_2(D_1, D_5) = 2/4$. For the function $\phi_3(D_1, D_5) = 0.75$ because the difference in days between the dates of D_1 and D_5 is 30 days. And finally $\phi_4(D_1, D_5) = 1$ because the two jobs are requested in the same place, Baltimore.

We can therefore construct the following graph with the function in Eq. (1) where each node is a job offer and each arete is the relationship

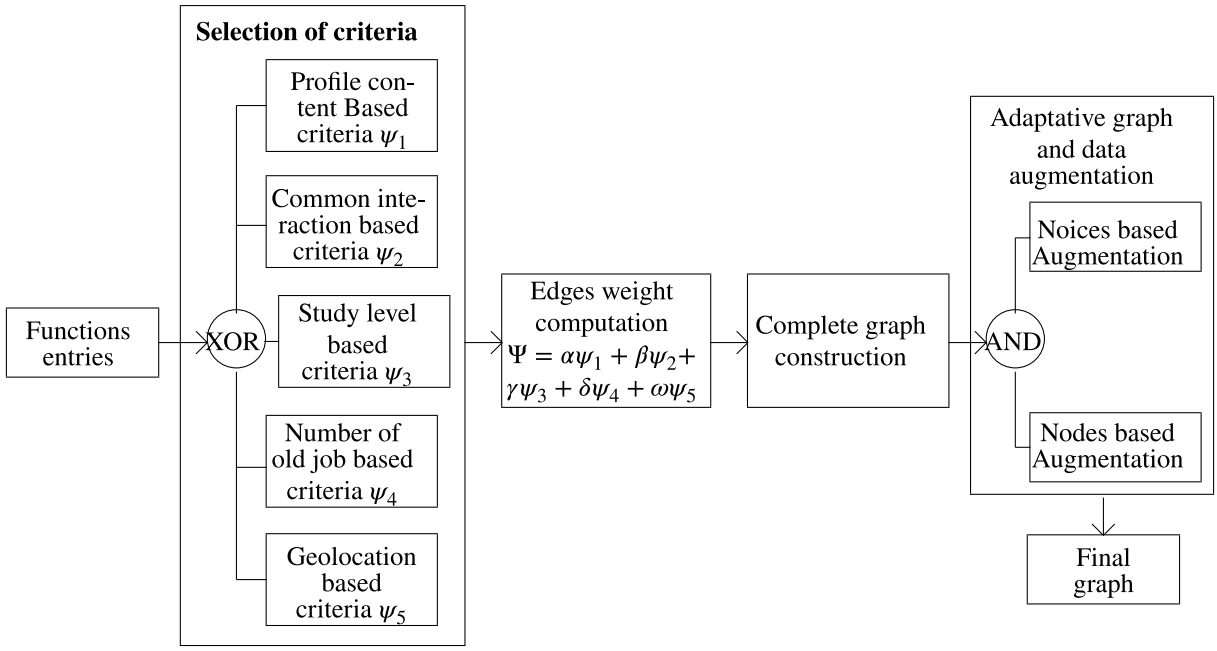

Fig. 3. Diagram of the construction of the job seeker-oriented graph based on the characteristic functions of job seekers.

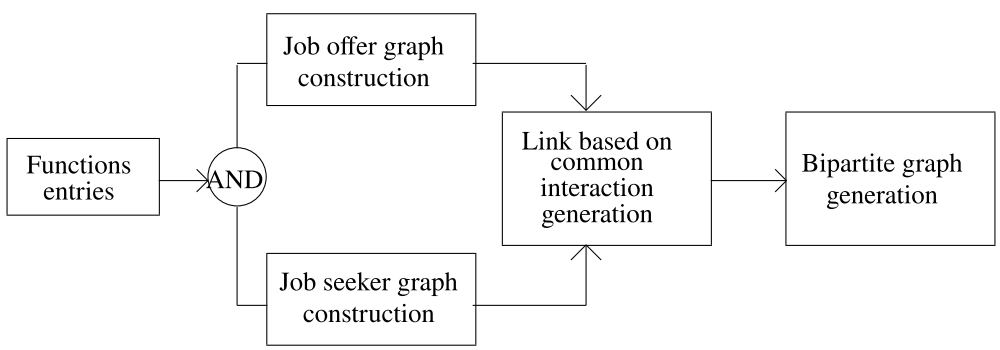

Fig. 4. Diagram showing the construction of the bipartite job offer and user graph.

Table 1
Extract of input functions for constructing the job offer graph.

Job_id	Descriptions	Users	Date	Location
J_1	*Network Administrator BA213015 nCorp Office in downtown...*	10, 15, 79, 2...	2012-04-30	Baltimore
J_2	*Financial Systems Coor-Orange County Sheriff Office...*	7, 25, 79, 21...	2012-05-10	Daytona
J_3	*Maus Chevrolet hiring: Experienced Transmission Technician...*	4, 5, 7, 25...	2012-04-05	Orlando
J_4	*TESL Adjunct-The Embry-Riddle Language Institute is seeking...*	1, 41, 47, 21...	2013-01-12	Orlando
J_5	*⟨b⟩FLORIDA CANCER SPECIALISTS SEEKS Research...*	10, 17, 19, 2...	2012-04-01	Baltimore
J_6	*⟨b⟩JENKINS SECURITY CONSULTANTS "We Take Secu...*	17, 33, 12, 1...	2012-05-10	Orlando

between these job offers, i.e. the extent to which they may be of interest to the same profile. Assuming that each of these criteria contributes the same amount to the calculation of the arrete's weight, so they all have the same weight of 0.25.

$$\Phi = 0.25\phi_1 + 0.25\phi_2 + 0.25\phi_3 + 0.25\phi_4 \tag{1}$$

In general, when constructing the graph, depending on the dataset, one may have all the characteristics or not. Some job offers may not necessarily include the location, or user profiles may not always have information on the level of education or other factors. The framework, in such cases, automatically adjusts the weights of the characteristics (the parameter values) to find the combination with the best performance. For characteristics that are absent, i.e., null, the selected weight will prioritize the characteristics that are present and ignore those that provide no information, particularly those that are already null (see Fig. 5).

In the first graph of Fig. 5 we have all the job offers and arretes weighted by our Φ function. At this stage, we have not yet performed any adaptive augmentation. In the second graph, a noise-based adaptive augmentation is performed, removing any arrears with a weight below the third quartile of the graph's arrears weights (in this case the value was 0.60). And finally, in the final graph, a node-based adaptive augmentation is performed, involving the deletion of neuds whose centrality is lower than the neud centrality whisker box and therefore of very low centrality, notably the J_2 node.

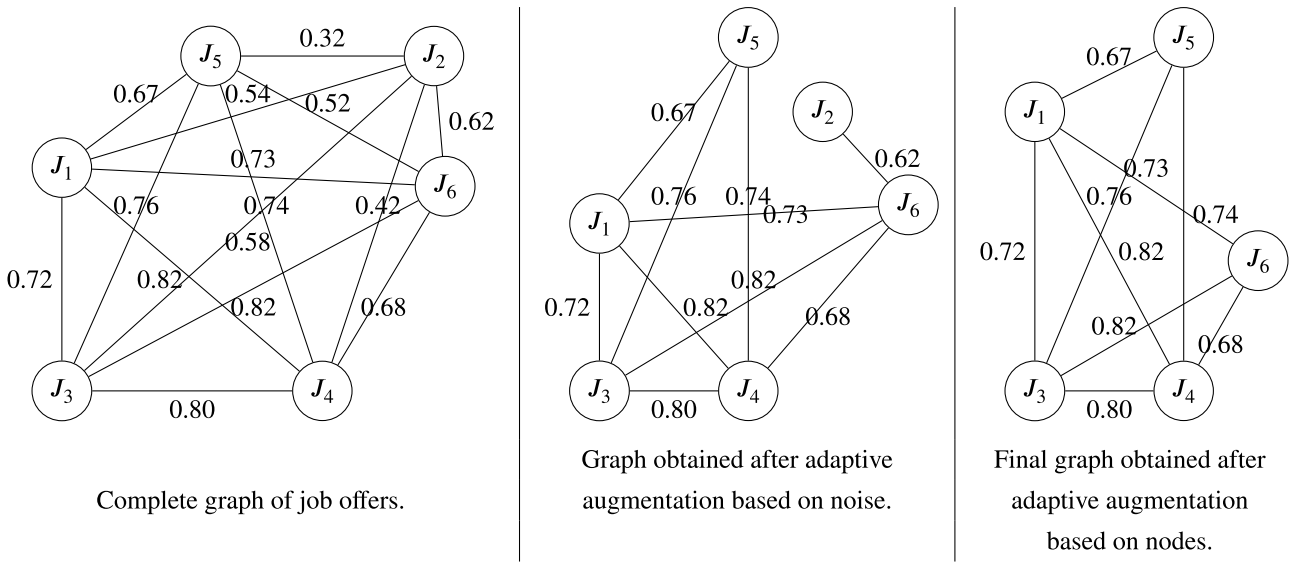

Fig. 5. Example of the construction of a graph of job offers, based on an extract of six job offers.

3.4. Framework models

Our framework proposes to build several graphs but also to carry out several recommendation schemes with various algorithms and strategies. We propose to compare the following 14 models:

- **KNN**: Here the strategy is to start directly from the graph and base the recommendation on the KNN algorithm (Suharyadi and Kusnadi, 2018). More explicitly, given a user U_i, to whom we wish to make a job recommendation, the following steps will be taken:
  - From the job offer graph: if the user has already interacted with a job offer, we will recommend the K offers closest in the job offer graph to the most recent offer with which he has interacted.
  - From the user-oriented graph: For datasets with job offers, we recommend the K offers closest to the offer with which its nearest neighbor has interacted. Otherwise (in the case of LinkedIn data) we make a CV recommendation as in the work of Wang et al. (2023).
  - From the bipartite graph: We recommend the K offers closest to him in this graph.
- **PageRank**: Here the principle of recommendation is the same as that with the **KNN** model, but instead of the KNN algorithm we use the PageRank random walk algorithm (Shalaby et al., 2017).
- **Node2Vec-DT/XGBoost/BERT4Rec/SVM/GCN**: Here we first embed the graph using the Node2Vec graph embedding method (Özlü et al., 2024), which is based on DeepWalk by introducing biased random walks exploring the graph in both width and depth, and capturing both the local and global structures of the graph. Based on this embedding, we then apply a machine learning algorithm (DT, XGBoost, BERT4Rec, SVM, GCN) to make recommendations.
- **GCN-DT/XGBoost/BERT4Rec/SVM/GCN**: Although GCN is used for semi-supervised learning on graph-structured data and is a more efficient version of convolutional neural networks. Based on the idea that neighboring nodes share similar labels, learn node representations by combining their own features with those of their neighbors. This process involves the aggregation of information from neighbors, a non-linear transformation of this information, and an iteration of these steps over several layers to capture global patterns. Finally, the learned representations are used to predict node labels. It is designed for feature extraction from graph data and therefore as an embedding algorithm, allowing these features to be used for node classification purposes. So, as with the previous models, we first perform an embedding, this time using the GCN method. From this embedding we then apply the decision tree algorithm to make recommendations (Yang et al., 2022). The GCN model is not only a feature extractor but also a semi-supervised learning model, which we will use in this case. Here the recommendation is made entirely with a GCN model inspired by work such as that of Wang et al. (2023), Yang et al. (2022) and Zhang and Xu (2024). In addition, in the case where GCN is both the graph embedding and recommendation technique, we use it directly on the graph without embedding, to avoid the bias of using the same method for both steps.
- **GraphSage-DT/XGBoost/BERT4Rec/SVM/GCN**: In this configuration, we first embed the graph using the graph embedding method GraphSage. Then, from this embedding, we apply the decision tree algorithm, XGboost, BERT4Rec, SVM and GCN to make recommendations.

4. Experimentation

In this section we will first present the data used in this work, then the experimental framework and the various parameters, and finally the protocol we used to evaluate the work.

4.1. Experimental data

To evaluate the performance of our framework, we use 02 popular job recommendation datasets available on kaggel. The first is a collection of 832,212 job offers and 389,709 users collected over the period 2010 to 2012. Each job offer consists of :

- an identifier: a unique integer for each job offer;
- a title: this is the job title;
- a description: this is a detailed presentation of the position for which the offer is submitted.
- the skills: this is a list of skills that summarizes the job description.
- Location: this is the country, region and city where the position is sought;
- The date: date of publication and end of validity of the offer.

While each user in the dataset is defined by :

Table 2
Nigham dataset and size of graph constructed.

Dataset	Number of jobs	Number of users	Type of graph	Number of nodes	Number of edges
Nigham	832 212	389 709	-Job oriented graph	580 096	1 980 576
			-User oriented graph	228 308	1 003 848
			-Bipartite graph	969 805	3 688 823

Table 3
LinkedIn dataset and size of graph constructed.

Dataset	Number of users	Type of graph	Number of nodes	Number of edges
LinkedIn	2 985 414	-User oriented graph	2 606 530	4 007 988

- An identifier: a unique integer that identifies the user;
- Skill list: a list of qualifications possessed by a user;
- Diplome : this is the level of study possessed by the job seeker;
- Interactions: this is a history of the interactions this user has had with the various job offers.
- Locations: this is the country, region and city where the user is located.

The second is a random fraction of a very large set of LinkedIn profiles (2,985,414) published by Zhang et al. (2015) and also used by Wang et al. (2023) . Each profile is made up of several pieces of information such as skills, educational and professional backgrounds, etc. Inspired by the formalization made in the work of Wang et al. (2023), we consider each profile on the LinkedIn platform as a user CV. To establish links between CVs, we use the 'also viewed' attribute, which is a list of other CVs viewed by users who have viewed this CV. The 'also viewed' attribute indicates the existence of a potential relationship between these CVs thanks to LinkedIn's underlying algorithm. The recommendation is therefore mainly made to so-called 'early' CVs, CVs of young graduates who have not yet had any professional experience.

4.2. Experimental settings and evaluation protocol

Our framework was implemented in Python, using the Jupyter and Google Collabs IDEs, and the code is available on GitHub.[5] Data preprocessing was carried out on an Ubuntu 22.04.1 LTS system equipped with a 64-bit core i5 processor with 16 GB RAM and 6 cores in Python language. To train the models, we opted to use a dedicated server with the following specifications: an Ubuntu 18.04.6 system, 64 GB of RAM and 64 cores. Also on Google collabs T4 GPU and 16 Go of RAM.

To assess the robustness of our framework, we randomly split the datasets into three sets, 60% for training, 20% for parameter validation, and 20% for performance testing. A cross-validation consisting of repeated random fragmentation (10 times) of the data for our experiments was also carried out. The proposed framework was used to construct the graphs with the characteristics shown in Tables 2 and 3.

The metrics for evaluating the performance of this framework, inspired by the previous works (Wang et al., 2023; Qin et al., 2023; Borisyuk et al., 2024), are as follows:

- **Accuracy**: This is a score that measures the prediction accuracy of an algorithm, by dividing the number of correct predictions by the number of predictions. Its formula is as follows:

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{2}$$

avec TP : True Positive, TN: True Negative, FP: False Positive and FN: False Negative.

- **Precision**: Precision is a score that provides information about the effectiveness of the recommendation model (Qin et al., 2023) . A model will have high precision if it recommends all relevant items while recommending as few as possible. Top k accuracy is a measure of the fraction of all relevant items among the first k items recommended. Its formula is as follows :

$$Precision(R)@K = \frac{\|r \in R : r \leq K\|}{K} \tag{3}$$

where K is the number of recommended items, R is the set of all recommendations and r is the relevant recommendations.

- **F1-score**: This is a harmonic mean between precision and recall. It is defined according to the formula (Qin et al., 2023) :

$$F1 - Score@K = \frac{2 \times Precision@K \times Recall@K}{Precision@K + Recall@K} \tag{4}$$

- **AUC**: The AUC for Area Under roc Curve is the probability that a relevant item taken at random from the list of recommendations is ranked higher than another irrelevant item in the same list (Borisyuk et al., 2024). A high probability implies a high AUC score, and therefore a better recommendation system. This probability is calculated empirically for each pair (relevant item, irrelevant item) taken from the list of recommendations provided by the ranking algorithm. The AUC score is therefore the proportion of well-ranked pairs over the rest of the pairs.

$$AUC(R)_K = \frac{1}{|R|(n - |R|)} \sum_{r \in R, r' \in (1,\ldots,n)/R} \sigma(r \leq r') \tag{5}$$

where $\frac{1}{|R|(n-|R|)}$ is out of all possible pairs of relevant and irrelevants items, the first sum is the rank of relevant item, the second sum is the rank of irrelevant and $\sigma(r \leq r')$ is a function that equals 1 if rank of relevant item is high than irrelevant items and 0 otherwise.

4.3. Results and comments

In this sub-section we will present the best results in turn, then the results as a function of parameter combinations and finally a comparison with existing results.

4.3.1. Best results

The Tables 4–6 present the results of our experiments on the Nigham and LinkedIn datasets. The tables are made up of the models used on the rows (e.g. Node2Vec-SVM refers to the SVM algorithm combined with the Node2Vec embedding technique, or GCN-DT which is the combination of graph embedding with the GCN technique and the decision tree algorithm). The columns contain the following evaluation metrics: Acc for Accuracy, Prec for Precision, F1 for F1-score and AUC for Under roc Curve. The values in the tables are the empirical averages of the best results for different combinations of parameter values after ten runs on randomly selected fractions of the data. The tables are divided into four blocks, the first of which contains the results obtained with methods based on graph traversal (KNN and PageRank) to which the other three blocks are compared. In bold we have the results that are better than those in the first block, and a star in front of the best result for each metric.

[5] https://github.com/ghislainWabo/recsys_for_job_offer.git.

- **Case of the job offer oriented graph**
  We begin with the results obtained by constructing only the job offer-oriented graph. In this case we only used the Nigham dataset because the LinkedIn data did not contain any job offers. The following observations can be made from Table 4:
  Observation from the models' point of view: Looking at Table 4, we can see that the first block containing results from graph traversal-based models (KNN and PageRank) performs less well than the other three blocks. We can also see that each model has at least the same performance as KNN and PageRank. And above all we can see that using GCN as a model always delivers the best performance (Accuracy of 94% corresponding to an improvement rate of 4. 44%, Accuracy of 96% corresponding to a rate of improvement of 5.49% compared with the basic models (KNN and PageRank), F1-score of 92% corresponding to a rate of improvement of 5.75%, AUC of 93% corresponding to a rate of improvement of 5.61%). He is followed by the SVM and XGboost models, which also deliver better performance, and finally DT and BERT4Rec.
  Observation from the point of view of the graph embedding technique: Depending on the graph embedding technique used (Node2Vec, GrapheSage, GCN), we can see that the performance values are fairly constant in the first two blocks, i.e. Node2Vec and GraphSage. However, the GCN technique stands out positively from the others because it is with this technique that the DT model, which stands out the least, obtains its greatest results, i.e. 93% accuracy and 95% precision.
  In short, we found that using GCN as a graph embedding and recommendation model technique, we obtained the best performance with improvement rates of between 4.44% and 5.75% depending on the evaluation metrics.
- **Case of job seeker oriented graph**
  Here the experiments were carried out on the two datasets Nigham and LinkedIn and the best results are recorded in Table 5 and allow us to make the following observations:
  Observation from the models' point of view: : The results in Table 5 show that although the graph traversal-based models (KNN and PageRank) performed well with accuracy levels reaching 91%, they still performed less well than the learning models. Indeed, the GCN model stands out with the best performance, regardless of the embedding technique used, with improvement rates of 7.14%, 6.9%, 11.25% and 8.54% respectively for the Accuracy, Accuracy, F1-score and AUC metrics. The next best performer is SVM, followed by XGboost, DT and BERT4Rec.
  Observation from the point of view of the graph embedding technique: Based on the graph embedding techniques (Node2Vec, GraphSage, GCN), we can see that the values of the results are fairly constant in each case, although the GCN technique stands out positively from the others because it is with this technique that the DT model which stands out the least obtains its greatest results, i.e. 92% accuracy.
  Observation from the point of view of the datasets : : It is quite obvious that the LinkedIn dataset gives poorer results than the Nigham dataset. The Accuracy values for the LinkedIn dataset range from 84% to 90%, while those for the Nigham dataset range from 90% to 93%. This may be justified by the fact that the Nigham dataset provided a larger learning base and more details on offers and users than LinkedIn's.
  In the same way as with the job offer directed graph above, on the results of the job seeker directed graph we note that using GCN as the graph embedding and recommendation model technique, we obtain the best performances with improvements up to 11.25%.
- **Case of bipartite graph**
  In this case we present the results of experiments obtained by constructing a bipartite graph consisting of a job offer-oriented part and a user-oriented part. This case was only carried out on the nigham dataset, and the results recorded in Table 6 allow us to make the following observations:
  Observation from the models' point of view: From Table 6, we can see that generally speaking, as for the other previous graphs, the learning models (DT, SVM, XGboost, BERT4Rec and GCN) are better than KNN and PageRank. Also in each of the last three blocks, the lines using GCN as a model always perform better (Accuracy of 96% corresponding to an improvement rate of 3.22%, Accuracy of 97% corresponding to an improvement rate of 3.19%, F1-score of 94% corresponding to an improvement rate of 3.3%, AUC of 95% corresponding to an improvement rate of 3.26%), followed by the SVM, XGboost and BERT4Rec models, which also perform better than DT.
  Observation from the point of view of the graph embedding technique: Depending on the graph embedding technique used (Node2Vec, GrapheSage, GCN), we can see that the performance values are fairly constant in the first two blocks, i.e. Node2Vec and GraphSage. However, the GCN technique stands out positively from the others because it is with this technique that the DT model, which stands out the least, obtains its best results, i.e. 96% accuracy and 97% precision.
  As with the previous job offer and job seeker oriented graphs, on the bipartite graph results we see that using GCN as the graph embedding and recommendation model technique, we get the highest performance with improvements up to 3% for accuracy, precision, F1-score and AUC.

Table 4
Tables of the best results obtained on the Nigham dataset in the case of the job offer oriented graph.

Datasets	Nigham			
Models	Acc	Prec	F1	AUC
KNN	0.90	0.91	0.87	0.89
PageRank	0.92	0.93	0.88	0.88
Node2Vec-DT	0.91	0.93	0.89↑	0.90↑
Node2Vec-GCN	0.93↑	0.94↑	0.90 ↑	0.90 ↑
Node2Vec-SVM	0.91	0.94↑	0.89 ↑	0.90↑
Node2Vec-XGboost	0.91	0.93	0.90 ↑	0.90↑
Node2Vec-BERT4Rec	0.92	0.93	0.89 ↑	0.90↑
GraphSage-DT	0.91	0.93	0.89↑	0.89
GraphSage-GCN	0.92	0.93	0.90↑	0.90↑
GraphSage-SVM	0.91	0.93	0.90↑	0.90↑
GraphSage-XGboost	0.91	0.92	0.89↑	0.90↑
GraphSage-BERT4Rec	0.90	0.91	0.89↑	0.89
GCN	**0.94***	**0.96***	**0.92***	**0.93***
GCN-DT	0.93↑	0.95↑	0.91↑	0.91↑
GCN-SVM	**0.94***	0.95 ↑	**0.92***	0.92↑
GCN-XGboost	0.93↑	0.95↑	**0.92***	0.92↑
GCN-BERT4Rec	0.92	0.94↑	0.91↑	0.91↑

Note: ↓ : decrease in performance, ↑ : increase in performance compared to KNN and PageRank and * is the best result compared to all the others.

4.3.2. Summary of results from a graph viewpoint

We can finally say that in general, the GCN model stands out by always giving us the best results. We can also see that from the point of view of graph types, the bipartite graph is the one that provides better performance than user-oriented graphs or job offer-oriented graphs. This is because the bipartite graph represents and contains more information and relationships than the other graphs. In Table 7 shows a comparison of the results according to the different graphs constructed.

We can see that with the bipartite graph of job offers and job seekers, we obtain better results on all the metrics than with the other graphs (job offer graphs and job seeker graphs). This can be justified by the fact that the bipartite graph contains information from both the job offer graph and the job seeker graph, as well as the relationships between the entities in the two graphs. This makes it more complete and may justify the learning being more accurate on this graph than on the others.

Table 5
Tables of the best results obtained on the Nigham and LinkedIn datasets in the case of the job seeker oriented graph.

Datasets	Nigham				LinkedIn			
Models	Acc	Prec	F1	AUC	Acc	Prec	F1	AUC
KNN	0.90	0.92	0.88	0.89	0.84	0.87	0.80	0.82
PageRank	0.91	0.92	0.87	0.87	0.84	0.88	0.80	0.83
Node2Vec-DT	0.92↑	0.94↑	0.89↑	0.91↑	0.84	0.88	0.82↑	0.83
Node2Vec-GCN	**0.93***	0.94↑	0.89↑	0.91↑	0.88↑	0.91↑	0.83↑	0.88↑
Node2Vec-SVM	0.92↑	**0.95***	0.90↑	0.90↑	0.87↑	0.90↑	0.83↑	0.87↑
Node2Vec-XGboost	0.91	0.94↑	0.90↑	0.91↑	0.87↑	0.89↑	0.82↑	0.86↑
Node2Vec-BERT4Rec	0.91	0.93↑	0.89↑	0.90↑	0.85↑	0.88	0.81↑	0.84↑
GraphSage-DT	0.92↑	**0.95***	0.90↑	0.90↑	0.84	0.89↑	0.82↑	0.83
GraphSage-GCN	**0.93***	**0.95***	0.91↑	**0.92***	0.89↑	0.91↑	0.84↑	0.88↑
GraphSage-SVM	0.92↑	0.94↑	0.91↑	0.91↑	0.87↑	0.91↑	0.84↑	0.86↑
GraphSage-XGboost	0.92↑	0.94↑	0.90↑	0.90↑	0.87↑	0.90↑	0.83↑	0.86↑
GraphSage-BERT4Rec	0.92↑	0.93↑	0.89↑	0.89	0.85↑	0.89↑	0.81↑	0.84↑
GCN	**0.93***	**0.95***	**0.92***	**0.92***	**0.90***	**0.93***	**0.89***	**0.89***
GCN-DT	0.92↑	0.94↑	0.91↑	0.90↑	0.87↑	0.91↑	0.84↑	0.88↑
GCN-SVM	**0.93***	**0.95***	**0.92***	0.91↑	0.89↑	0.92↑	0.88↑	0.87↑
GCN-XGboost	**0.93***	**0.95***	0.91↑	0.90↑	0.89↑	0.92↑	0.88↑	0.87↑
GCN-BERT4Rec	**0.93***	0.93↑	0.90↑	0.90↑	0.86↑	0.90↑	0.85↑	0.84↑

Note: ↓ : decrease in performance, ↑ : increase in performance compared to KNN and PageRank and * is the best result compared to all the others.

Table 6
Tables of the best results obtained on the Nigham dataset in the case of the bipartite graph.

Datasets	Nigham			
Models	Acc	Prec	F1	AUC
KNN	0.93	0.94	0.91	0.92
PageRank	0.93	0.94	0.92	0.92
Node2Vec-DT	0.92 ↓	0.94	0.91	0.91↓
Node2Vec-GCN	0.94↑	0.95↑	0.92	0.93↑
Node2Vec-SVM	0.94↑	0.95↑	0.92	0.93↑
Node2Vec-XGboost	0.93	0.94	0.91	0.92
Node2Vec-BERT4Rec	0.93	0.95↑	0.92	0.92
GraphSage-DT	0.93	0.94	0.91	0.92
GraphSage-GCN	0.95↑	0.96↑	0.92	0.94↑
GraphSage-SVM	0.94 ↑	0.95↑	0.92	0.93↑
GraphSage-XGboost	0.94↑	0.94	0.92	0.93↑
GraphSage-BERT4Rec	0.94↑	0.95↑	0.92	0.93↑
GCN	**0.96***	**0.97***	**0.94***	**0.95***
GCN-DT	0.93	0.95↑	0.90↓	0.91↓
GCN-SVM	0.95↑	0.96↑	0.93↑	0.94↑
GCN-XGboost	0.95↑	0.96↑	0.93↑	0.93↑
GCN-BERT4Rec	0.94↑	0.96↑	0.93↑	0.93↑

Note: ↓ : decrease in performance, ↑ : increase in performance compared to KNN and PageRank and * is the best result compared to all the others.

Table 7
Comparison of graph results.

Models	GCN			
Graphs	Acc	Prec	F1	AUC
Bipartite graph	**0.96***	**0.97***	**0.94***	**0.95***
Job offer oriented graph	0.94	0.96	0.92	0.93
Job seeker oriented graph	0.93	0.95	0.92	0.92

4.3.3. Impact of parameters on results

- **Case of Job offer oriented graph:** The Table 8 presents the combinations of parameters (feature weights) that led to the best results for the Nigham corpus with the job offer graph. The values are quadruplets (4-uplets) $(\alpha, \beta, \gamma, \delta)$, which are the feature weights that led to the best performances. For example, the quadruplet (0.5, 0.25, 0, 0.25) gave the highest Accuracy value for the PageRank model, which allows us to say that features based on the content of the offer description were important at 50%, those based on shared interactions and geolocation were each important at 25% and the temporal feature was not taken into account in this case.
  The following observations can be made from this table:
  The 4-uplets (0.5, 0.5, 0, 0) give the best performance in 26/68 cases: so in 38.24% of cases the characteristics based on the content of the offers and on their common interactions are the most important, with a contribution of 50% each.
  The 4-uplets (0.25, 0.25, 0.25, 0.25) gives the best performance in 19/68 cases : to 27.94% of cases all the features contribute 25% each. We can also see that it is this combination that gives us the highest accuracy values.
  The 4-uplets (0.5, 0.25, 0.25, 0) enables us to obtain the best performances in 14/68 cases: thus at 20.59% of cases the descriptions of the offers contribute to 50% followed by the common interactions and the temporal characteristics which contribute to 25% each.
  The 4-uplets (0.5, 0.25, 0, 0.25) achieves the best performance in 9/68 cases: here in 13.24% of cases the location-based feature replaces the time-based feature in the previous combination.
  Furthermore, from the point of view of the models, we observed that the DT, SVM and XGboost models provide better results when the first two parameters are at 0.5 and the last two are at 0, i.e. are not taken into account. In other words, for these models, the description of the job offers and the common interactions on these offers are sufficient to obtain their best results. On the other hand, the GCN model works well when all the parameters are at 0.25, i.e. when all the characteristics are taken into account.
  From these results we can see that 38% of the best results are obtained when the date and location characteristics are not taken into account, and that when we take all the characteristics into account we obtain 27.94% of the best results. Moreover, when we first remove the location-based feature and then the date-based feature, we find that we obtain more better results 20.59% with the time-based feature than with the location-based feature, which only obtains 13.24% of the best results.
- **Case of job seeker oriented graph:** The Table 9 shows us the combinations of parameters (feature weights) which gave the best results for the Nigham corpus with the user graph. For each cell in the table there is a 5-tuple $(\alpha, \beta, \gamma, \delta, \omega)$, which represents the combination of weights that obtained the best performance. For example, the combination: (0.4, 0.4, 0, 0.2, 0) gave the highest Accuracy value for the KNN model on the user graph, which allows us to say that the characteristics based on the content of the profile, and on common interactions were also more important in this case than the characteristics based on the number of jobs

Table 8
Combination of the parameters that provided the best performance in the form of 4-uplets $(\alpha, \beta, \gamma, \delta)$ representing the respective weight of job descriptions, common interactions on job offers, temporal dynamics and location with the job offer oriented graph on the Nigham dataset.

Datasets	Nigham			
Models	Acc	Prec	F1	AUC
KNN	(0.5, 0.5, 0, 0)	(0.25, 0.25, 0.25, 0.25)	(0.5, 0.25, 0.25, 0)	(0.5, 0.25, 0.25, 0)
PageRank	(0.5, 0.25, 0, 0.25)	(0.25, 0.25, 0.25, 0.25)	(0.25, 0.25, 0.25, 0.25)	(0.25, 0.25, 0.25, 0.25)
Node2Vec-DT	(0.5, 0.5, 0, 0)	(0.5, 0.5, 0, 0)	(0.5, 0.25, 0, 0.25)	(0.5, 0.25, 0.25, 0)
Node2Vec-GCN	(0.25, 0.25, 0.25, 0.25)	(0.25, 0.25, 0.25, 0.25)	(0.25, 0.25, 0.25, 0.25)	(0.25, 0.25, 0.25, 0.25)
Node2Vec-SVM	(0.5, 0.5, 0, 0)	(0.5, 0.5, 0, 0)	(0.5, 0.25, 0.25, 0)	(0.5, 0.25, 0.25, 0)
Node2Vec-XGboost	(0.5, 0.5, 0, 0)	(0.5, 0.5, 0, 0)	(0.5, 0.25, 0, 0.25)	(0.5, 0.25, 0.25, 0)
Node2Vec-BERT4Rec	(0.5, 0.5, 0, 0)	(0.5, 0.25, 0.25, 0)	(0.5, 0.25, 0, 0.25)	(0.5, 0.25, 0.25, 0)
GraphSage-DT	(0.5, 0.5, 0, 0)	(0.5, 0.5, 0, 0)	(0.5, 0.25, 0, 0.25)	(0.5, 0.25, 0.25, 0)
GraphSage-GCN	(0.25, 0.25, 0.25, 0.25)	(0.25, 0.25, 0.25, 0.25)	(0.25, 0.25, 0.25, 0.25)	(0.25, 0.25, 0.25, 0.25)
GraphSage-SVM	(0.5, 0.5, 0, 0)	(0.5, 0.5, 0, 0)	(0.5, 0.5, 0, 0)	(0.5, 0.25, 0.25, 0)
GraphSage-XGboost	(0.5, 0.5, 0, 0)	(0.5, 0.5, 0, 0)	(0.5, 0.25, 0, 0.25)	(0.5, 0.25, 0.25, 0)
GraphSage-BERT4Rec	(0.5, 0.5, 0, 0)	(0.5, 0.5, 0, 0)	(0.5, 0.25, 0, 0.25)	(0.5, 0.5, 0, 0)
GCN	(0.25, 0.25, 0.25, 0.25)	(0.25, 0.25, 0.25, 0.25)	(0.5, 0.5, 0, 0)	(0.25, 0.25, 0.25, 0.25)
GCN-DT	(0.5, 0.5, 0, 0)	(0.25, 0.25, 0.25, 0.25)	(0.5, 0.25, 0, 0.25)	(0.5, 0.25, 0.25, 0)
GCN-SVM	(0.5, 0.5, 0, 0)	(0.25, 0.25, 0.25, 0.25)	(0.5, 0.5, 0, 0)	(0.5, 0.25, 0.25, 0)
GCN-XGboost	(0.5, 0.5, 0, 0)	(0.25, 0.25, 0.25, 0.25)	(0.5, 0.25, 0, 0.25)	(0.5, 0.25, 0.25, 0)
GCN-BERT4Rec	(0.5, 0.5, 0, 0)	(0.25, 0.25, 0.25, 0.25)	(0.5, 0.5, 0, 0)	(0.5, 0.5, 0, 0)

Table 9
Combination of the parameters that provided the best performance in the form of 5-uplets $(\alpha, \beta, \gamma, \delta, \omega)$ representing the respective weight of profile descriptions, common interactions on job offers, study level, number of old job and location with the job seeker oriented graph on the Nigham dataset.

Datasets	Nigham			
Models	Acc	Prec	F1	AUC
KNN	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)
PageRank	(0.4, 0.4, 0, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.2, 0.2, 0.2, 0.2, 0.2)
Node2Vec-DT	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.2, 0.2, 0.2, 0.2, 0.2)
Node2Vec-GCN	(0.4, 0.4, 0, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.2, 0.2, 0.2, 0.2, 0.2)
Node2Vec-SVM	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.2, 0.4, 0.2, 0.2, 0)
Node2Vec-XGboost	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)
Node2Vec-BERT4Rec	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)
GraphSage-DT	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.2, 0.2, 0.2, 0.2, 0.2)
GraphSage-GCN	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)
GraphSage-SVM	(0.4, 0.4, 0, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.2, 0.2, 0.2, 0.2, 0.2)
GraphSage-XGboost	(0.4, 0.4, 0, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.2, 0.2, 0.2, 0.2, 0.2)
GraphSage-BERT4Rec	(0.4, 0.4, 0, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.2, 0.2, 0.2, 0.2, 0.2)
GCN	(0.4, 0.4, 0, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)
GCN-DT	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)
GCN-SVM	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.2, 0.2, 0.2, 0.2, 0.2)
GCN-XGboost	(0.2, 0.4, 0.2, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)
GCN-BERT4Rec	(0.4, 0.4, 0, 0.2, 0)	(0.4, 0.4, 0, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)	(0.2, 0.4, 0.2, 0.2, 0)

currently held; unlike level of education and geolocation, which were not taken into account in this case.

The following observations can be made:

The 5-uplets (0.4, 0.4, 0, 0.2, 0) gives the best performance in 27/68 cases : This means that in 39.71% of cases the characteristics based on the user's profile content and on their common interactions are the most important, each at 40%, and that the one based on the number of jobs previously held is also important, but at 20%, and that finally the two other characteristics (based on the level of education and based on geolocation) are not.

The 5-uplets (0.2, 0.4, 0.2, 0.2, 0) gave the best performance in 23/68 cases : This means that in 33.82% of cases the feature based on users' common interactions is the most important with a weighting of 40%, and that those based on the content of the user's profile, on the number of jobs previously held and on the level of education are also important but with a weighting of 20% each and that finally the feature based on geolocation is not.

The 5-tuple (0.2, 0.2, 0.2, 0.2, 0.2) gives the best performance in 18/68 cases : This means that in 26.47% of cases all the characteristics are equally important, with a weighting of 20% each.

We can also observe that for the DT, SVM, XGboost and BERT4Rec models, the best results are obtained when the feature based on common interactions is more important, the feature based on geolocation is neglected and the others have a weight of 0.2. However, with the GCN model it is preferable for the features based on profile content and common interactions to have the same weight (0.4), the number of experiences to be 0.2 and the others to be neglected.

From the results presented above on the combinations of parameters, we note that 39.71% of the best results are obtained when the characteristics based on level of education and location are not taken into account, and that when only the characteristic based on location is not taken into account we obtain 33.82% of the best results. Furthermore, when all the characteristics are taken into account at the same time, we find that we obtain 26.47% better results.

Table 10
Comparison tables of the results obtained with the results of the work of Wang et al. (2023), Yang et al. (2022) and Wu et al. (2024).

Datasets	Nigham				LinkedIn			
Models	Acc	Prec	F1	AUC	Acc	Prec	F1	AUC
GLFJRec	**0.93**	**0.95**	**0.92**	**0.92**	**0.90**	**0.93**	**0.89**	**0.89**
GLRec (Wu et al., 2024)	0.90	0.91	0.81	0.84	0.89	0.90	0.81	0.85
IHGCN (Wang et al., 2023)	***	***	***	***	0.859	0.901	0.822	0.848
CGA2T (Yang et al., 2022)	0.923	***	0.596	***	***	***	***	***
Improve. ↑	3.33%	4.4%	54.4%	9.52%	5.88%	3.33%	8.88%	5.95%

4.3.4. General discussion

The GCN model, based on performance, has outperformed the other models used (BERT4Rec, SVM, XGBoost, and Decision Trees). This can be justified by its ability to handle complex relationships in the data. Unlike these models, which generally process data in isolation, GCN is designed to work with graph-structured data, capturing subtle relationships between users, job offers, and their interactions. GCN excels at propagating information across the graph, allowing for the recommendation of relevant items even in the absence of direct interactions. More effective for non-linear and nested data structures, unlike models such as SVM, XGBoost, and Decision Trees, it is also more robust to missing data, as it can use information from neighboring nodes to make predictions. Finally, GCN learns embeddings for each node, which allows it to more effectively capture implicit preferences and complex relationships than models like BERT4Rec, which focuses on sequential interactions, or traditional methods like SVM and Decision Trees that rely on explicit features.

Furthermore, we also justify the good performance of the GCN technique from an embedding perspective due to its ability to effectively capture complex and global relationships, propagate information through the graph, model non-linear dependencies, and integrate contextual information more robustly.

Finally, the framework offers great flexibility in constructing its recommendation system and parameters, depending on the dataset used.

4.3.5. Comparison with previous work

In order to position ourselves in relation to the literature, we proposed to make a comparison with previous works that have used the data available to us. As we did not have access to their code, we tested our framework on it. As a result, the results obtained have been compared in Table 10 with those of the works by Yang et al. (2022) and Wang et al. (2023). Also having had the GLRec framework from the work of Wu et al. (2024) (available on GitHub[6]) at our disposal but not the data they used, we proposed to run it on our two datasets and compare the results.

In Table 10 we compare the results obtained with our GLFJRec framework with those of previous works (IHGCN by Wang et al., 2023 and CGA2T by Yang et al., 2022) on Nigham and LinkedIn data. We have marked in bold for each case the best result, so the table shows that it is the GLFJRec line that has the best results. For the Nigham dataset, GLFJRec obtains an increase of 1% for the accuracy metric and 32.4% for the F1-score metric compared to CGA2T. Furthermore, for the LinkedIn dataset, GLFJRec provided an improvement of 5% for the accuracy metric, 3% for the precision metric, 7% for the F1-score metric and 5% for the AUC metric compared to IHGCN. And compared to the GLRec framework, we get a 3% increase in the accuracy metric, a 4% increase in the precision metric, an 11% increase in the F1-score metric and an 8% increase in the AUC metric on the Nigham dataset and a 1% increase in the accuracy metric, a 3% increase in the precision metric, an 8% increase in the F1-score metric and a 4% increase in the AUC metric on the LinkedIn dataset.

[6] https://github.com/WLiK/GLRec.

5. Conclusion and future work

In this work, we addressed the problem of graph-based job recommendation by proposing a general framework based on graph learning for recommender systems named GLFJRec. It is a framework that firstly allows graphs to be constructed from the main characteristics of job offers and job seekers, where the weights of the characteristics can vary so that they can be adapted according to the dataset. It can then be used for graph learning to obtain a graph embedding representation, and finally it can also be used for graph learning to make recommendations directly on the graph or from graph embeddings. To do this, we construct three graphs (job offer-oriented graphs, job seeker-oriented graphs and bipartite graphs of job seekers and job offers) from characteristics based on the description of the job offer or job seeker profile, the interactions between job offers and job seeker profiles, the temporal dynamics, based on the location, the level of education of the job seeker profiles, and on the number of experiences of the job seeker profiles. We assign weights to these characteristics $\alpha, \beta, \gamma, \delta, \omega$, which take their values in the interval $[0, 1]$ with for condition the sum must be equal to 1. For the embedding we used GraphSage, Node2Vec, GCN and for the learning models Decision Tree, XGboost, SVM, BERT4Rec and GCN for the computation of the recommendations.

The experiments were carried out on two datasets (Nigham and LinkedIn) with the construction of three types of graphs (job offer oriented graph, job seekers oriented graph and bipartite graph of job offers and job seekers) for the Nigham dataset and only one graph (job seekers oriented) for the LinkedIn dataset. For the job offer oriented graph, we had four parameters (the characteristic based on the content of the offer: description of the offer, the characteristic based on the common interactions carried out on the offers: clicks and applications carried out by the same profile on two offers, the characteristic based on the temporal dynamics: time taken since the publication of the offer and the characteristic based on the location: the place where the profile of the offer will be exercised. For the job seeker oriented graph we have five (in addition to the characteristics based on the content of the profiles, the common interactions between profiles and the location of the profiles, we have the characteristics based on the level of the job seeker and finally the one based on the number of professional experiences already carried out), and for the bipartite graph we have used the four from the job offer oriented graph and the five from the job seeker oriented graph.

We ran a total of 35 experiments that we repeated ten times on randomly drawn fractions of the data for cross-validation. This enabled us to obtain results reaching 96% for accuracy, 97% for precision, 93% for F1-score and 94% for AUC, which corresponds respectively to 7%, 5%, 11% and 11% of improvements compared with standard models based on graph traversal (KNN and PageRank). And finally we compared ourselves to the work of Wang et al. (2023), Yang et al. (2022) and that of Wu et al. (2024) and obtained better performance each time with an increase up to 54% for F1-score.

This work has demonstrated that we can recommend job offers using a framework that allows us to employ strategies based on various models, and even add other models, with different techniques that can be adjusted as needed. Additionally, it allows for parameter tuning to achieve better performance than existing methods. However, some limitations can be identified: First, the quality of the data plays a crucial role in the effectiveness of the recommendation system, and poor-quality data could affect the relevance of the recommendations generated. Furthermore, scaling this framework to larger datasets could present challenges in terms of scalability, potentially requiring optimizations such as parallelization to reduce execution time and maintain high performance at scale. Finally, a thorough study of the interpretability of the recommendations generated by this framework would be beneficial to better understand its decision-making process and ensure transparency and user trust. As part of future work, we plan to enhance this framework to determine the most optimal graph for a given dataset and to compare different graph embeddings in order to select the most suitable embedding method for each problem.

CRediT authorship contribution statement

Ghislain Wabo Tatchum: Writing – original draft, Methodology, Formal analysis, Data curation, Conceptualization. **Armel Jacques Nzekon Nzeko'o:** Validation, Formal analysis, Conceptualization. **Thomas Messi Nguele:** Validation. **Xaveria Youh Djam:** Validation, Supervision.

Ethical and informed consent for data used

The authors declare that in this article, the two datasets used are free and pose no ethical conflicts because they were acquired on open access platforms. Their use therefore poses no ethical problems.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The code and the cleaned data set are available from the corresponding author by request.

References

Afandi, M., Isnaini, K.N., 2024. Analyzing public trust in presidential election surveys: A study using SVM and logistic regression on social media comments. J. Comput. Sci. Eng. (JCSE) 5 (1), 1–11.

Akter, S., Nawal, N., Dey, A., Das, A., 2023. Analyzing the IT job market and classifying IT jobs using machine learning algorithms. In: Applied Intelligence for Industry 4.0. Chapman and Hall/CRC, pp. 240–252.

Borisyuk, F., He, S., Ouyang, Y., Ramezani, M., Du, P., Hou, X., Jiang, C., Pasumarthy, N., Bannur, P., Tiwana, B., et al., 2024. Lignn: Graph neural networks at linkedin. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 4793–4803.

Chou, Y.-C., Yu, H.-Y., 2020. Based on the application of AI technology in resume analysis and job recommendation. In: 2020 IEEE International Conference on Computational Electromagnetics. ICCEM, IEEE, pp. 291–296.

Dave, V.S., Zhang, B., Al Hasan, M., AlJadda, K., Korayem, M., 2018. A combined representation learning approach for better job and skill recommendation. In: Proceedings of the 27th ACM International Conference on Information and Knowledge Management. pp. 1997–2005.

Freire, M.N., de Castro, L.N., 2021. e-Recruitment recommender systems: A systematic review. Knowl. Inf. Syst. 63 (1), 1–20. http://dx.doi.org/10.1007/s10115-020-01522-8.

Giabelli, A., Malandri, L., Mercorio, F., Mezzanzanica, M., Seveso, A., 2021. Skills2Job: A recommender system that encodes job offer embeddings on graph databases. Appl. Soft Comput. 101, 107049.

Han, X., Zhu, C., Hu, X., et al., 2024. Adapting job recommendations to user preference drift with behavioral-semantic fusion learning. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 1004–1015.

Hjort, J., Poulsen, J., 2019. The arrival of fast internet and employment in Africa. Am. Econ. Rev. 109 (3), 1032–1079.

Jiang, M., Fang, Y., Xie, H., Chong, J., Meng, M., 2019a. User click prediction for personalized job recommendation. World Wide Web 22 (1), 325–345.

Jiang, M., Fang, Y., Xie, H., Chong, J., Meng, M., 2019b. User click prediction for personalized job recommendation. World Wide Web 22, 325–345.

Jin, L., Zhang, L., Zhao, L., 2023. Feature selection based on absolute deviation factor for text classification. Inf. Process. Manage. 60 (3), 103251.

Joy, S.P., et al., 2021. A job recommendation system for differently abled using neural networks. TURCOMAT 12 (10), 2751–2762.

Luo, H., Ma, S., Selvaraj, A.J.B., Sun, Y., 2019. Learning job representation using directed graph embedding. In: Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data. pp. 1–5.

Mhamdi, D., Moulouki, R., El Ghoumari, M.Y., Azzouazi, M., Moussaid, L., 2020. Job recommendation based on job profile clustering and job seeker behavior. Procedia Comput. Sci. 175, 695–699.

Nigam, A., Roy, A., Singh, H., Waila, H., 2019. Job recommendation through progression of job selection. In: 2019 IEEE 6th International Conference on Cloud Computing and Intelligence Systems. IEEE, Ccis, pp. 212–216.

Özlü, Ö.A., Orman, G.K., Turhan, S.N., 2024. Exploring graph-based techniques in job recommendation systems. In: 2024 IEEE 12th International Conference on Intelligent Systems. IS, IEEE, pp. 1–6.

Patel, R., Mishra, S., Ailani, N., Mirani, S., Ren, Z., Mohammed, S., 2024. Personalized context-oriented job recommendation system based on knowledge graph. Authorea Prepr..

Pelánek, R., 2020. Measuring similarity of educational items: An overview. IEEE Trans. Learn. Technol. 13 (2), 354–366. http://dx.doi.org/10.1109/TLT.2019.2896086.

Perkins, J., 2010. Python Text Processing With NLTK 2.0 Cookbook. PACKT publishing.

Qin, C., Zhu, H., Shen, D., Sun, Y., Yao, K., Wang, P., Xiong, H., 2023. Automatic skill-oriented question generation and recommendation for intelligent job interviews. ACM Trans. Inf. Syst. 42 (1), 1–32.

Shalaby, W., AlAila, B., Korayem, M., Pournajaf, L., AlJadda, K., Quinn, S., Zadrozny, W., 2017. Help me find a job: A graph-based approach for job recommendation at scale. In: 2017 IEEE International Conference on Big Data. Big Data, IEEE, pp. 1544–1553.

Suharyadi, J., Kusnadi, A., 2018. Design and development of job recommendation system based on two dominants on psychotest results using KNN algorithm. IJNMT (Int. J. New Media Technol.) 5 (2), 116–120.

Suvankulov, F., Chi Keung Lau, M., Ho Chi Chau, F., 2012. Job search on the internet and its outcome. Internet Res. 22 (3), 298–317.

Tatchum, G.W., Nzeko'o, A.J.N., Makembe, F.S., Djam, X.Y., 2024. Class-oriented text vectorization for text classification: Case study of job offer classification. J. Comput. Sci. Eng. (JCSE) 5 (2), 116–136.

Uysal, A.K., Gunal, S., 2014. The impact of preprocessing on text classification. Inf. Process. Manage. 50 (1), 104–112.

Wang, Y., Shi, K., Niu, Z., 2020. A session-based job recommendation System Combining Area knowledge and interest graph neural networks. In: SEKE. pp. 489–492.

Wang, H., Yang, W., Li, J., Ou, J., Song, Y., Chen, Y., 2023. An improved heterogeneous graph convolutional network for job recommendation. Eng. Appl. Artif. Intell. 126, 107147.

Wisdom, V., Gupta, R., 2016. An Introduction to Twitter Data Analysis in Python. Artigence Inc.

Wu, L., Qiu, Z., Zheng, Z., Zhu, H., Chen, E., 2024. Exploring large language model for graph data understanding in online job recommendations. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 38, pp. 9178–9186, 8.

Yang, Y., Miao, R., Wang, Y., Wang, X., 2022. Contrastive graph convolutional networks with adaptive augmentation for text classification. Inf. Process. Manage. 59 (4), 102946.

Zhang, X., Ma, H., Yang, F., Li, Z., Chang, L., 2023. KGCL: A knowledge-enhanced graph contrastive learning framework for session-based recommendation. Eng. Appl. Artif. Intell. 124, 106512. http://dx.doi.org/10.1016/j.engappai.2023.106512, URL: https://www.sciencedirect.com/science/article/pii/S0952197623006966.

Zhang, Y., Tang, J., Yang, Z., Pei, J., Yu, P.S., 2015. Cosnet: Connecting heterogeneous social networks with local and global consistency. In: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1485–1494.

Zhang, Q., Xu, Y., 2024. Knowledge graph embedding with inverse function representation for link prediction. Eng. Appl. Artif. Intell. 127, 107225.

Zheng, Z., Qiu, Z., Hu, X., et al., 2023. Generative job recommendations with large language model. arXiv preprint arXiv:2307.02157.

Ghislain Wabo Tatchum is a Ph.D. candidate at the computer Science Department of the University of Yaoundé 1. His main research interests are recommender systems, machine learning and graph theory.

Armel Jacques Nzekon Nzeko'o received the Ph.D. degree, from Sorbone Université and University of Yaoundé 1 in 2019. He is currently a Senior lecturer at University of Yaoundé 1. His principal research interests are on topics related to recommender systems, machine learning and parallel machine learning.

Thomas Messi Nguele received the Ph.D. degree, from University of Yaoundé 1 in 2018. He is currently Head of Department Computer engineering at University of Ebolowa and Senior lecturer at University of Yaoundé 1. His principal research interests are on topics related to parallel machine learning, machine learning and Parallel recommender systems.

Xaviera Youh Djam received the Ph.D. degree, from University of Yaoundé 1 in 2014. She is currently a Senior lecturer and Senior Software Engineer at University of Yaoundé 1. Her principal research interests are on topics related to Software engineering, Software testing and Artificial Intelligence Applications.